

LAPORAN
TUGAS 5 DOCKER IMPLEMENTASI ANALISIS INDEKS MASSA
TUBUH (BMI) BERBASIS WEB MENGGUNAKAN DOCKER DAN
AWS

Disusun untuk Memenuhi Tugas Mata Kuliah Komputasi Awan



Disusun oleh

15-2023-142 - Irfan Nur Iqbal

Prodi Informatika
Fakultas Teknologi Industri
Institut Teknologi Nasional
Bandung
2026

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi yang semakin pesat telah membuka peluang besar dalam pengolahan data kesehatan secara digital dan berbasis jaringan. Salah satu indikator sederhana namun penting dalam dunia kesehatan masyarakat adalah Indeks Massa Tubuh (Body Mass Index/BMI). BMI merupakan metode standar yang digunakan secara global untuk mengkategorikan kondisi berat badan seseorang berdasarkan perbandingan antara berat badan (kilogram) dan kuadrat tinggi badan (meter).

Dengan memanfaatkan teknologi web modern dan layanan komputasi awan (cloud computing), proses perhitungan serta analisis BMI dapat dilakukan secara otomatis, real-time, dan dapat diakses oleh siapa saja melalui jaringan internet. Hal ini memberikan nilai tambah yang signifikan dalam hal aksesibilitas dan skalabilitas layanan kesehatan berbasis digital.

Pada tugas ini, dikembangkan sebuah aplikasi web bernama VitalTrack BMI yang mampu melakukan analisis BMI secara interaktif. Aplikasi ini diimplementasikan menggunakan teknologi Docker untuk containerization serta layanan cloud Amazon Web Services (AWS), khususnya Elastic Compute Cloud (EC2) untuk deployment frontend dan Elastic Container Service (ECS) untuk deployment backend.

1.2 Tujuan Penulisan

Tujuan dari implementasi aplikasi ini adalah sebagai berikut:

- Mengembangkan aplikasi berbasis web yang mampu menghitung dan menganalisis nilai Indeks Massa Tubuh (BMI) pengguna.
- Mengimplementasikan backend REST API menggunakan Node.js dan framework Express.
- Menggunakan SQLite sebagai sistem manajemen basis data untuk menyimpan riwayat data pengguna.
- Mengemas aplikasi backend ke dalam container menggunakan teknologi Docker.
- Melakukan deployment aplikasi pada infrastruktur cloud AWS, yaitu EC2 untuk frontend dan ECS Fargate untuk backend.
- Mendokumentasikan seluruh proses implementasi sebagai panduan teknis yang dapat direplikasi.

BAB II

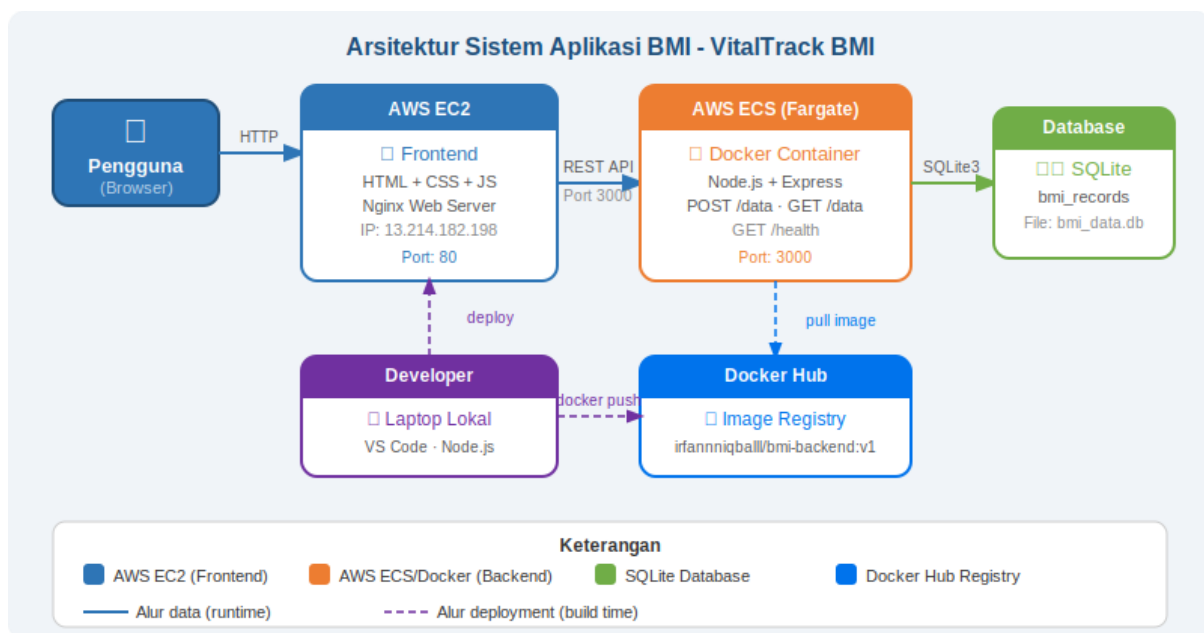
ARSITEKTUR SISTEM

2.1 Deskripsi Arsitektur

Frontend berfungsi sebagai lapisan presentasi yang berinteraksi langsung dengan pengguna. Komponen ini dikembangkan menggunakan HTML5, CSS3, dan JavaScript murni tanpa framework tambahan, kemudian di-hosting pada instance AWS EC2 menggunakan web server Nginx. Backend berperan sebagai lapisan logika bisnis yang menangani seluruh pemrosesan data, perhitungan BMI, dan komunikasi dengan database. Komponen ini dikembangkan menggunakan runtime Node.js dengan framework Express dan dijalankan di dalam container Docker pada layanan AWS ECS Fargate. Database SQLite digunakan sebagai sistem penyimpanan data yang ringan dan efisien untuk menyimpan riwayat analisis BMI pengguna.

2.2 Diagram Arsitektur

Diagram di bawah ini menggambarkan alur komunikasi antar komponen sistem. Garis solid menunjukkan alur data saat runtime, sedangkan garis putus-putus menunjukkan alur deployment. Pengguna mengakses frontend melalui browser, frontend berkomunikasi dengan backend melalui REST API di port 3000, dan backend menyimpan serta mengambil data dari database SQLite.



2.3 Teknologi yang Digunakan

Komponen	Teknologi	Keterangan
Frontend	HTML5, CSS3, JavaScript	Antarmuka pengguna berbasis web
Web Server	Nginx	Melayani file statis frontend di EC2
Backend	Node.js + Express.js	REST API dan logika bisnis

Komponen	Teknologi	Keterangan
Database	SQLite3	Penyimpanan data riwayat BMI
Containerization	Docker (image Alpine)	Pengemasan backend ke container
Registry	Docker Hub	Penyimpanan Docker image
Cloud Frontend	AWS EC2 (Ubuntu 22.04)	Hosting frontend
Cloud Backend	AWS ECS Fargate	Menjalankan container backend

BAB III

IMPLEMENTASI FRONTEND

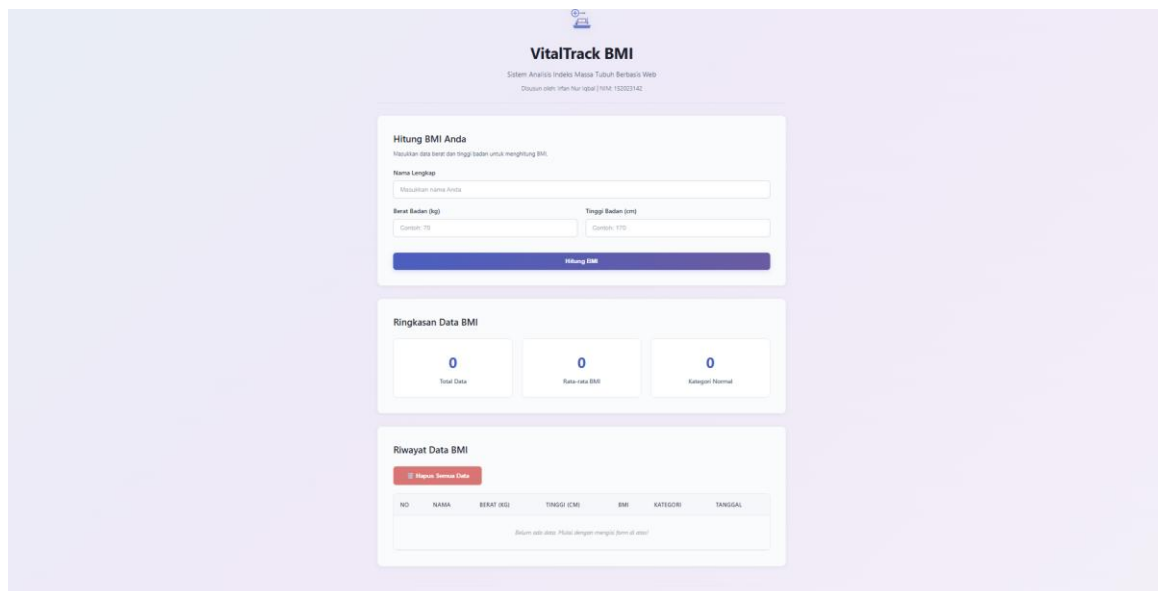
3.1 Deskripsi Frontend

Frontend aplikasi VitalTrack BMI dikembangkan menggunakan teknologi web standar, yaitu HTML5, CSS3, dan JavaScript tanpa menggunakan framework JavaScript tambahan. Pendekatan ini dipilih untuk menjaga kesederhanaan dan portabilitas aplikasi. Antarmuka dirancang agar profesional, responsif, dan informatif sehingga memudahkan pengguna dalam memasukkan data serta memahami hasil analisis BMI yang ditampilkan.

3.2 Fitur-Fitur Frontend

Fitur-fitur yang tersedia pada antarmuka frontend aplikasi ini antara lain:

- Form input data pengguna yang memuat kolom nama lengkap, berat badan (dalam kilogram), dan tinggi badan (dalam sentimeter).
- Tampilan hasil analisis BMI secara langsung setelah formulir diajukan, meliputi nilai BMI, kategori kesehatan, dan pesan rekomendasi.
- Statistik ringkasan yang menampilkan total data, rata-rata nilai BMI, dan jumlah pengguna dalam kategori Normal.
- Tabel riwayat data seluruh pengguna yang disimpan dalam database, diurutkan berdasarkan waktu input terbaru.
- Tombol hapus data untuk mereset seluruh riwayat yang tersimpan di database.
- Indikator loading dan notifikasi error untuk meningkatkan pengalaman pengguna.



Gambar 3.1 Tampilan Antarmuka Utama Aplikasi VitalTrack BMI

3.3 Pengujian Input dan Output

Pengujian input dan output dilakukan dengan memasukkan data berat badan dan tinggi badan yang bervariasi untuk memastikan aplikasi dapat menghitung nilai BMI secara akurat serta menampilkan kategori kesehatan yang tepat.

The screenshot displays a web application interface for BMI calculation. At the top, there is a navigation bar with various links. Below it, a form titled 'Hitung BMI' contains input fields for 'Nama' (Name), 'Berat Badan' (Weight), and 'Tinggi Badan' (Height). The 'Berat Badan' field is set to '55 kg' and the 'Tinggi Badan' field is set to '165 cm'. Below these fields, the 'Nilai BMI' (BMI Value) is calculated as '20.2' and the 'Kategori' (Category) is 'Normal'. A 'Pesan Kesehatan' (Health Message) box indicates that the user is in good health. Below the analysis results, there is a 'Ringkasan Data BMI' (BMI Data Summary) section showing '1' total data, a BMI value of '20.2', and a 'Normal' category. At the bottom, there is a 'Riwayat Data BMI' (BMI Data History) section with a table listing the calculation history.

ID	NAMA	BERAT (KG)	TINGGI (CM)	BMI	KATEGORI	TANGGAL
1	Irfan Nur Iqbal	55	165	20.2	Normal	11/04/2024 16:37

Gambar 3.2 Tampilan Hasil Analisis BMI (Contoh: Berat 55 kg, Tinggi 165 cm → BMI 20.2 → Normal)

Berdasarkan pengujian yang dilakukan, aplikasi mampu menerima input dari pengguna, mengirimkan data ke backend, menerima hasil perhitungan, dan menampilkan hasil analisis BMI beserta kategori kesehatan secara real-time.

BAB IV

IMPELEMENTASI BACKEND

4.1 Deskripsi Backend

Backend aplikasi dikembangkan menggunakan runtime Node.js dengan framework Express.js. Backend berperan sebagai otak sistem yang menerima permintaan dari frontend melalui HTTP, melakukan validasi data, menghitung nilai BMI, menyimpan hasil ke database SQLite, serta mengembalikan respons dalam format JSON. Backend juga menerapkan middleware CORS (Cross-Origin Resource Sharing) untuk mengizinkan komunikasi lintas origin.

4.2 Endpoint API

API backend menyediakan tiga endpoint utama yang dapat diakses oleh frontend maupun klien lainnya:

Endpoint	Metode	Deskripsi	Request Body
/health	GET	Memeriksa status server (health check)	-
/data	POST	Menerima dan menyimpan data BMI baru	{ name, weight, height }
/data	GET	Mengambil seluruh riwayat data BMI	-
/data	DELETE	Menghapus seluruh data BMI dari database	-

4.3 Logika Perhitungan BMI

Perhitungan BMI dilakukan di sisi server (backend) menggunakan rumus standar WHO sebagai berikut:

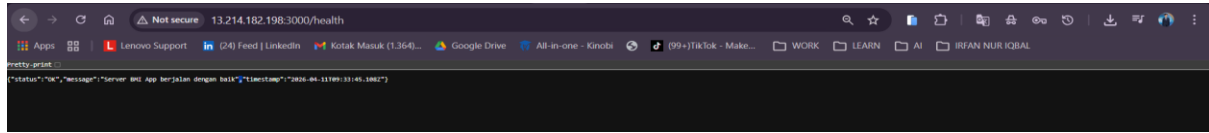
$$\text{BMI} = \text{Berat Badan (kg)} \div [\text{Tinggi Badan (m)}]^2$$

Setelah nilai BMI diperoleh, kategorisasi dilakukan berdasarkan standar WHO sebagai berikut:

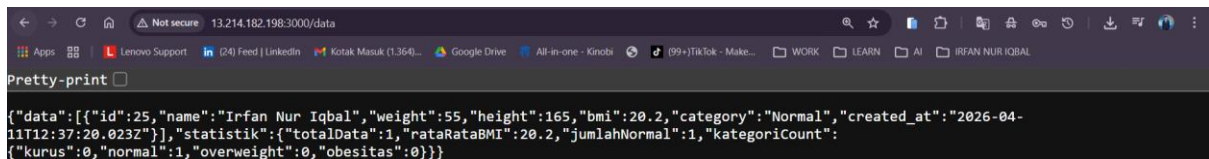
Rentang Nilai BMI	Kategori	Keterangan
< 18.5	Kurus (Underweight)	Berat badan berada di bawah rentang ideal
18.5 – 24.9	Normal	Indeks massa tubuh berada pada rentang ideal
25.0 – 29.9	Overweight	Berat badan berada di atas rentang ideal
≥ 30.0	Obesitas (Obese)	Risiko kesehatan tinggi, disarankan konsultasi dokter

4.4 Pengujian API

Pengujian API dilakukan menggunakan browser dengan mengakses endpoint /health untuk memverifikasi status server. Selain itu, pengujian endpoint /data dilakukan melalui interaksi langsung dari antarmuka frontend.



Gambar 4.1 Response JSON Endpoint /health: {"status": "OK", "message": "Server BMI App berjalan dengan baik"}



Gambar 4.2 Response JSON Endpoint /data Beserta Statistik

BAB V

IMPELEMENTASI DATABASE

5.1 Deskripsi Database

Database yang digunakan dalam aplikasi ini adalah SQLite, sebuah sistem manajemen basis data relasional berbasis file yang bersifat serverless. SQLite dipilih karena sifatnya yang ringan, tidak memerlukan proses server database terpisah, mudah dikonfigurasi dalam lingkungan container Docker, dan memiliki performa yang memadai untuk skala aplikasi ini. File database disimpan di dalam container pada path ./database/bmi_data.db.

5.2 Struktur Tabel

Database memiliki satu tabel utama bernama bmi_records yang digunakan untuk menyimpan seluruh data analisis BMI pengguna. Definisi tabel menggunakan SQL berikut:

```
sqlite> .tables
bmi_records
sqlite> .schema bmi_records
CREATE TABLE bmi_records (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  name TEXT NOT NULL,
  weight REAL NOT NULL,
  height REAL NOT NULL,
  bmi REAL NOT NULL,
  category TEXT NOT NULL,
  created_at TEXT NOT NULL
);
sqlite>
```

Berikut adalah penjelasan masing-masing kolom pada tabel bmi_records:

Nama Kolom	Tipe Data	Constraint	Keterangan
id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Identifikasi unik setiap record
name	TEXT	NOT NULL	Nama pengguna yang dimasukkan
weight	REAL	NOT NULL	Berat badan dalam satuan kilogram (kg)
height	REAL	NOT NULL	Tinggi badan dalam satuan sentimeter (cm)
bmi	REAL	NOT NULL	Nilai BMI hasil perhitungan (2 desimal)
category	TEXT	NOT NULL	Kategori kesehatan (Kurus/Normal/Overweight/Obesitas)
created_at	TEXT	NOT NULL	Timestamp ISO 8601 saat data disimpan

5.3 Pengujian Penyimpanan Data

Pengujian dilakukan dengan memasukkan beberapa data BMI melalui antarmuka frontend dan memverifikasi bahwa data tersebut berhasil disimpan serta dapat ditampilkan kembali pada tabel riwayat.

```
sqlite> SELECT * FROM bmi_records;  
1|Irfan Nur Iqbal|55.0|165.0|20.2|Normal|2026-04-11T07:54:25.435Z  
2|Iqbal|70.0|150.0|31.11|Obesitas|2026-04-11T08:06:41.143Z  
sqlite>
```

Gambar 5.1 Tabel Riwayat Data BMI yang Berhasil Disimpan ke Database SQLite

Hasil pengujian menunjukkan bahwa setiap data yang dimasukkan melalui form berhasil disimpan ke dalam database SQLite dan dapat diambil kembali untuk ditampilkan pada halaman riwayat aplikasi.

BAB VI

IMPLEMENTASI DOCKER

6.1 Deskripsi Container

Docker digunakan untuk mengemas seluruh backend aplikasi beserta dependensinya ke dalam sebuah container yang dapat dijalankan secara konsisten di berbagai lingkungan (environment), baik di mesin pengembang lokal maupun di server cloud AWS. Penggunaan container memberikan keuntungan berupa isolasi lingkungan, portabilitas, dan kemudahan deployment serta replikasi.

Image Docker yang digunakan berbasis node:18-alpine, yaitu image Node.js versi 18 yang dibangun di atas distribusi Alpine Linux, dipilih karena ukurannya yang sangat kecil (sekitar 45 MB) dibandingkan image Node.js standar. Hal ini mempercepat proses build, push, dan pull image.

6.2 Konfigurasi Dockerfile

Konfigurasi Dockerfile yang digunakan untuk membangun image backend adalah sebagai berikut:

```
# Gunakan Node.js Alpine sebagai base image (ringan, ~45MB)
FROM node:18-alpine
```

```
# Set working directory di dalam container
WORKDIR /app
```

```
# Copy package.json dan package-lock.json terlebih dahulu
# (memanfaatkan layer caching Docker untuk efisiensi build)
COPY package*.json ./
```

```
# Install hanya dependency production
RUN npm install --production
```

```
# Copy seluruh source code backend ke container
COPY . .
```

```
# Buat direktori database jika belum ada
RUN mkdir -p database
```

```
# Expose port 3000 untuk komunikasi API
EXPOSE 3000
```

```
# Set environment ke production
ENV NODE_ENV=production
```

```
# Health check menggunakan Node.js (tanpa curl)
HEALTHCHECK --interval=30s --timeout=3s --start-period=5s --retries=3
\
  CMD node -e "require('http').get('http://localhost:3000/health',
(r) => {
  process.exit(r.statusCode === 200 ? 0 : 1);
}).on('error', () => process.exit(1))"

# Jalankan server saat container distart
CMD ["node", "server.js"]
```

Penjelasan setiap instruksi Dockerfile:

- FROM node:18-alpine: mendefinisikan base image yang ringan berbasis Alpine Linux dengan Node.js versi 18.
- WORKDIR /app: menetapkan /app sebagai direktori kerja di dalam container, sehingga semua perintah selanjutnya dijalankan dari direktori ini.
- COPY package*.json ./ + RUN npm install: menyalin file konfigurasi dependency dan menginstalnya. Dilakukan sebelum copy source code untuk memanfaatkan layer caching Docker.
- COPY . .: menyalin seluruh file source code backend ke dalam container.
- RUN mkdir -p database: memastikan direktori untuk file SQLite tersedia.
- EXPOSE 3000: mendokumentasikan bahwa container menggunakan port 3000.
- ENV NODE_ENV=production: mengatur environment variable untuk mode produksi.
- HEALTHCHECK: mendefinisikan pemeriksaan kesehatan container yang dijalankan secara berkala.
- CMD ["node", "server.js"]: perintah default yang dijalankan saat container dimulai.

6.3 Proses Build dan Run Container

Proses pembangunan dan pengoperasian container Docker dilakukan di server EC2 Ubuntu menggunakan perintah-perintah berikut:

Langkah 1 - Build Docker Image:

```
docker build -t bmi-backend .
```

Langkah 2 - Jalankan Container:

```
docker run -d -p 3000:3000 --name bmi-api bmi-backend
```

Langkah 3 - Verifikasi Container Berjalan:

```
docker ps
```

```

WARNING: The legacy builder is deprecated and will be removed in a future release.
Install the buildx component to build images with BuildKit:
https://docs.docker.com/go/buildx/

Sending build context to Docker daemon 110.6kB
Step 1/10 : FROM node:18-alpine
--> 8d6421d663b4
Step 2/10 : WORKDIR /app
--> Using cache
--> 410fd342fdd8
Step 3/10 : COPY package*.json ./
--> Using cache
--> 9d649b118358
Step 4/10 : RUN npm install --production
--> Using cache
--> 4abf258529a8
Step 5/10 : CMD npm run start
--> Using cache
--> 2642b7795fa9
Step 6/10 : RUN mkdir -p database
--> Using cache
--> d37b1d1d1ee2
Step 7/10 : EXPOSE 3000
--> Using cache
--> 4d899d2601ff
Step 8/10 : ENV NODE_ENV=production
--> Using cache
--> 832e64ff1e19
Step 9/10 : HEALTHCHECK --interval=30s --timeout=3s --start-period=5s --retries=3 CMD node -e "require('http').get('http://localhost:3000/health', (r) => { process.exit(r.statusCode === 200 ? 0 : 1); }).on('error', () => pr
process.exit(1))"
--> Using cache
--> f5a224472ed
Step 10/10 : CMD ["node", "server.js"]
--> Using cache
--> b0d1c3ab8464
Successfully built b0d1c3ab8464

```

Gambar 6.1 Output Proses Build Docker Image di EC2

```

ubuntu@ip-172-31-0-59:~/bmi-app/backend$ sudo docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
d4538f4fa348   bmi-backend   "docker-entrypoint.s..." 4 hours ago   Up 4 hours   0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp   bmi-api

```

Gambar 6.2 Daftar Container yang Sedang Berjalan (Status: Up, Healthy)

Hasil perintah docker ps menunjukkan container bmi-api berjalan dengan status Up dan Healthy. Port 3000 berhasil dipetakan (0.0.0.0:3000->3000/tcp), yang berarti backend dapat diakses dari luar container.

BAB VII

DEPLOYMENT KE AWS

7.1 Deployment Frontend (EC2)

Frontend aplikasi di-deploy pada instance Amazon EC2 menggunakan Ubuntu Server 22.04 LTS dengan tipe instance t2.micro (free tier eligible). Web server Nginx digunakan untuk melayani file-file statis frontend (HTML, CSS, JavaScript).

Konfigurasi instance EC2:

Parameter	Nilai
AMI	Ubuntu Server 22.04 LTS
Instance Type	t2.micro (1 vCPU, 1 GB RAM)
Storage	8 GB SSD
Security Group	SSH (22), HTTP (80), Custom TCP (3000)
Public IP	13.214.182.198
Region	ap-southeast-1 (Singapore)

Langkah-langkah deployment frontend ke EC2:

1. Buat dan jalankan instance EC2 baru dengan konfigurasi di atas.
2. Hubungkan ke instance menggunakan EC2 Instance Connect di browser.
3. Instal dan konfigurasi Nginx sebagai web server:

```
sudo apt update && sudo apt install nginx -y
```

```
sudo systemctl start nginx && sudo systemctl enable nginx
```

4. Salin file frontend ke direktori web Nginx:

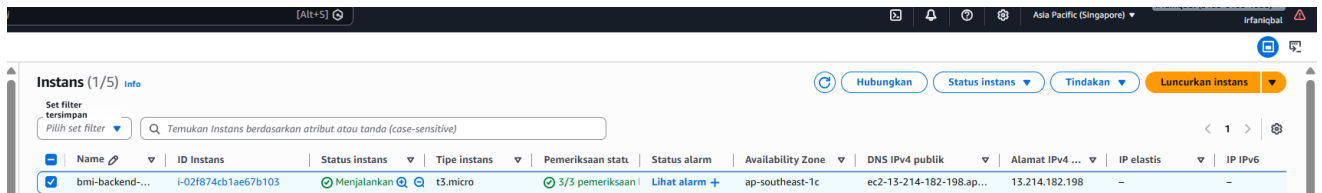
```
sudo cp index.html style.css script.js /var/www/html/
```

5. Pastikan API_BASE_URL pada script.js mengarah ke backend EC2:

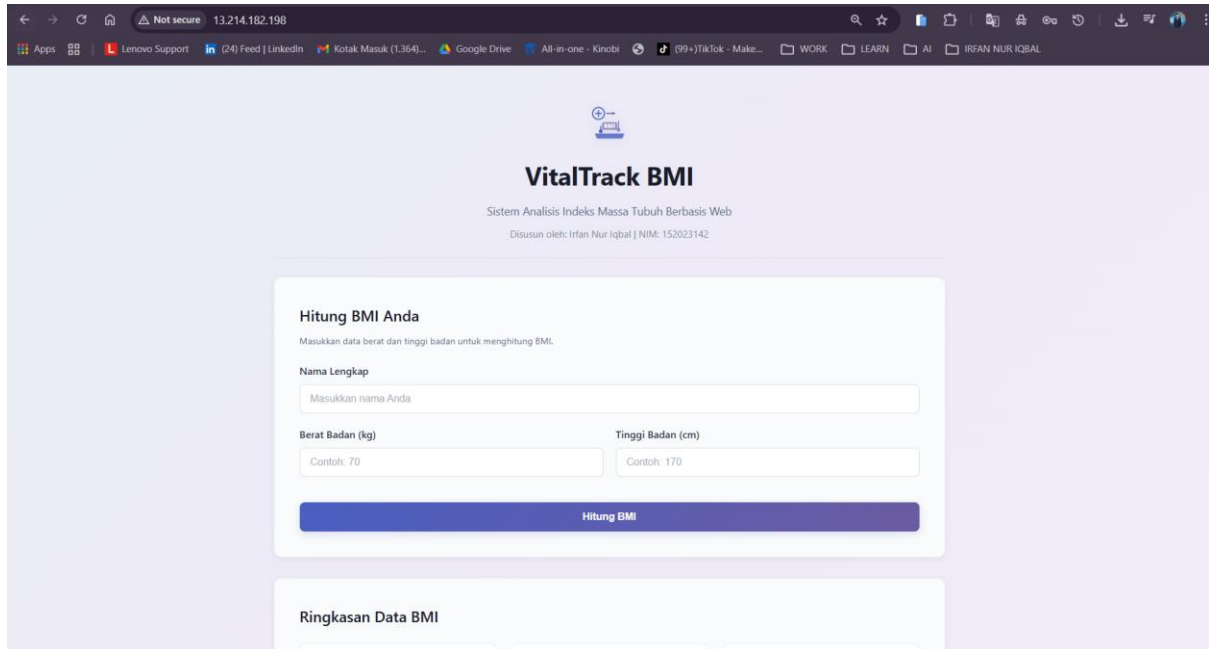
```
const API_BASE_URL = "http://13.214.182.198:3000";
```

6. Restart Nginx dan akses aplikasi melalui IP publik EC2.

```
sudo systemctl restart nginx
```



Gambar 7.1 Instance EC2 dengan Status Running



Gambar 7.2 Aplikasi Frontend Berjalan di Browser melalui EC2 (<http://13.214.182.198>)

7.2 Push Image ke Docker Hub

Sebelum melakukan deployment ke ECS, Docker image backend harus diunggah ke Docker Hub agar dapat diakses oleh layanan ECS. Berikut adalah langkah-langkah yang dilakukan:

1. Login ke Docker Hub dari terminal EC2:

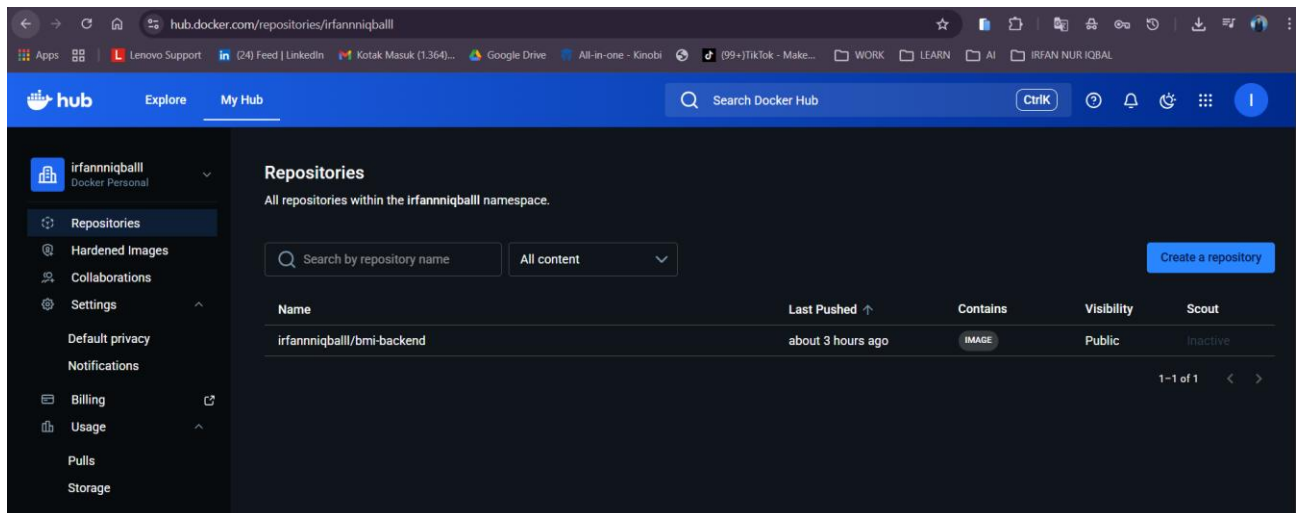
```
sudo docker login -u irfannniqballl
```

2. Berikan tag pada image dengan format username/repo:tag:

```
sudo docker tag bmi-backend irfannniqballl/bmi-backend:v1
```

3. Push image ke Docker Hub:

```
sudo docker push irfannniqballl/bmi-backend:v1
```



Gambar 7.3 Repository Docker Hub irfannniqballl/bmi-backend:v1 Berhasil Diunggah

7.3 Deployment Backend (ECS Fargate)

Backend container di-deploy menggunakan AWS Elastic Container Service (ECS) dengan launch type AWS Fargate. ECS Fargate merupakan layanan serverless yang memungkinkan penjalankan container tanpa harus mengelola infrastruktur server secara langsung.

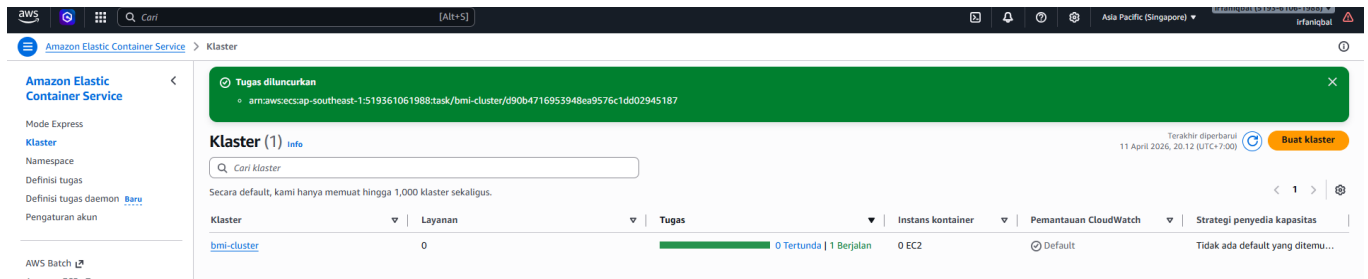
Konfigurasi ECS:

Parameter	Nilai
Cluster Name	bmi-cluster
Launch Type	AWS Fargate
Task Definition	bmi-backend
Container Image	irfannniqballl/bmi-backend:v1
Container Port	3000 (TCP)
CPU	0.25 vCPU
Memory	0.5 GB
Security Group	ecs-bmi-sg (Port 3000 terbuka)
Public IP	Aktif (Enabled)

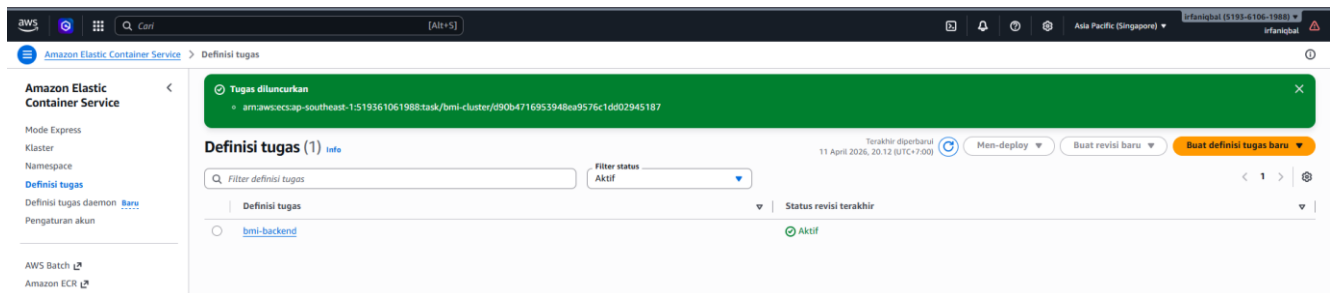
Langkah-langkah deployment backend ke ECS:

1. Buat ECS Cluster baru dengan nama bmi-cluster menggunakan Fargate.
2. Buat Task Definition dengan konfigurasi container menggunakan image dari Docker Hub.

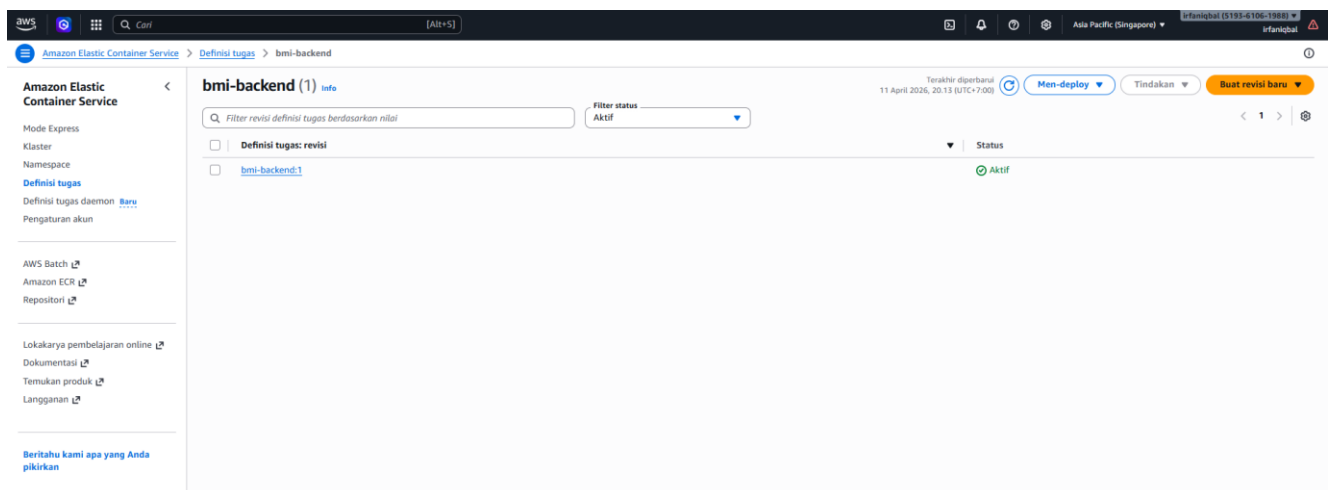
3. Jalankan task pada cluster dengan mengaktifkan Public IP.
4. Verifikasi task berstatus RUNNING dan endpoint dapat diakses.



Gambar 7.4 ECS Cluster bmi-cluster



Gambar 7.5 Task Definition bmi-backend dengan Image dari Docker Hub



Gambar 7.6 ECS Task berstatus RUNNING — Backend Berhasil Dijalankan di Cloud

BAB VIII

HASIL PENGUJIAN

8.1 Hasil Pengujian Fungsional

Setelah seluruh komponen berhasil diimplementasikan dan dideploy, dilakukan pengujian fungsional terhadap aplikasi secara menyeluruh. Pengujian mencakup seluruh fitur yang dipersyaratkan dalam spesifikasi tugas, mulai dari antarmuka pengguna, komunikasi API, penyimpanan data, containerization, hingga deployment cloud.

The screenshot displays the VitalTrack BMI web application. The top section, titled "VitalTrack BMI", describes it as a "Sistem Analisis Indeks Massa Tubuh Berbasis Web" developed by Irfan Nur Iqbal. Below this is a form titled "Hitung BMI Anda" where users input their name, weight, and height. The bottom section, "Hasil Analisis BMI", shows the calculated results for a user named Iqbal with a weight of 60 kg and height of 180 cm, resulting in a BMI of 18.52, categorized as "Normal". A health message states that the user's body mass index is in the ideal range.

VitalTrack BMI
Sistem Analisis Indeks Massa Tubuh Berbasis Web
Disusun oleh: Irfan Nur Iqbal | NIM: 152023142

Hitung BMI Anda
Masukkan data berat dan tinggi badan untuk menghitung BMI.

Nama Lengkap

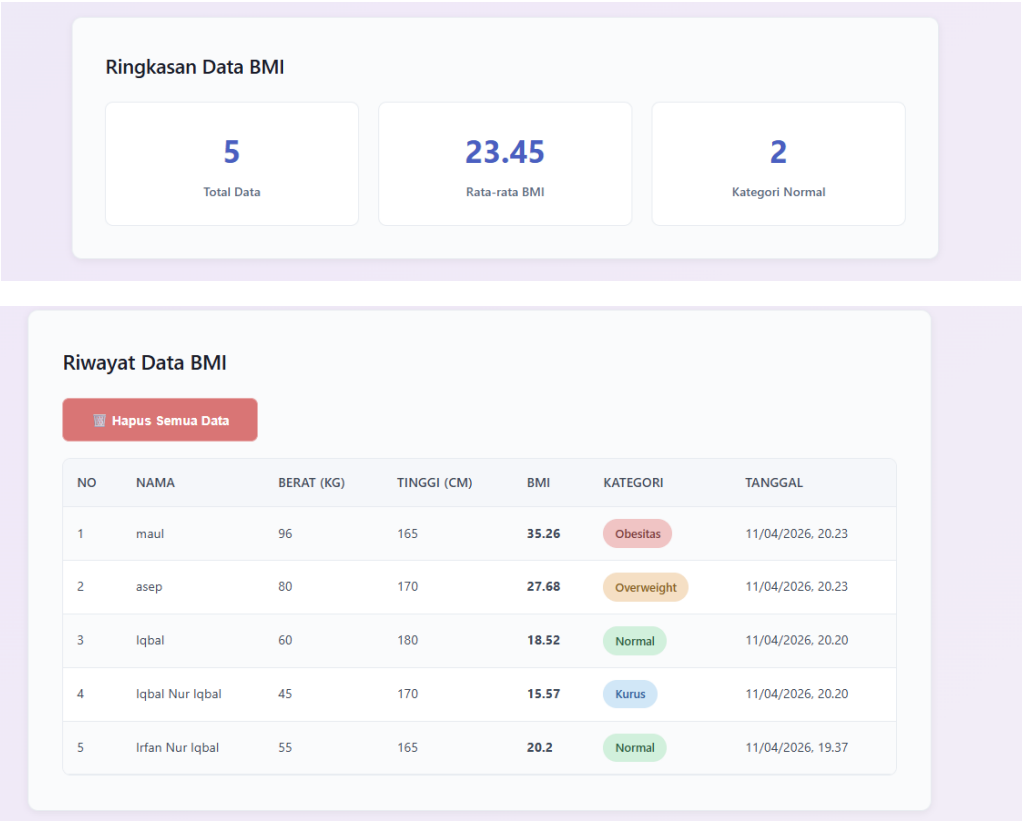
Berat Badan (kg) Tinggi Badan (cm)

Hitung BMI

Hasil Analisis BMI

Nama	Iqbal
Berat Badan	60 kg
Tinggi Badan	180 cm
Nilai BMI	18.52
Kategori	Normal

Pesan Kesehatan
Indeks massa tubuh berada pada rentang ideal.



8.2 Tabel Rekap Hasil Pengujian

No	Skenario Pengujian	Hasil yang Diharapkan	Status
1	Input data BMI (nama, berat, tinggi)	Data diterima oleh backend dan diproses	Berhasil
2	Perhitungan nilai BMI	Nilai BMI akurat sesuai rumus standar	Berhasil
3	Penentuan kategori BMI	Kategori tepat sesuai rentang WHO	Berhasil
4	Penyimpanan data ke SQLite	Data tersimpan di tabel bmi_records	Berhasil
5	Tampilan tabel riwayat data	Seluruh data ditampilkan dengan benar	Berhasil
6	Statistik ringkasan	Total, rata-rata BMI, jumlah Normal akurat	Berhasil
7	Endpoint GET /health	Mengembalikan status OK	Berhasil

No	Skenario Pengujian	Hasil yang Diharapkan	Status
8	Docker image ter-build	Image bmi-backend:latest tersedia	Berhasil
9	Docker container berjalan	Status container Up dan Healthy	Berhasil
10	Push image ke Docker Hub	Image tersedia di irfannniqbali/bmi-backend:v1	Berhasil
11	Frontend jalan di EC2	Halaman dapat diakses via IP publik	Berhasil
12	Backend jalan di ECS	Task ECS berstatus RUNNING	Berhasil

BAB IX

KESIMPULAN

Berdasarkan seluruh proses implementasi dan pengujian yang telah dilakukan, dapat ditarik kesimpulan sebagai berikut:

- Aplikasi VitalTrack BMI berbasis web telah berhasil dikembangkan secara lengkap dengan menggunakan HTML5, CSS3, dan JavaScript sebagai frontend, serta Node.js + Express sebagai backend, dan SQLite sebagai sistem basis data.
- Penggunaan Docker untuk containerization backend berhasil dilaksanakan dengan membangun image yang ringan berbasis node:18-alpine, menjalankannya sebagai container, serta memverifikasi fungsionalitasnya melalui health check.
- Image Docker backend berhasil dipublikasikan ke Docker Hub (irfannniqballl/bmi-backend:v1) sebagai registry terpusat yang dapat diakses oleh layanan cloud AWS ECS.
- Deployment frontend berhasil dilakukan pada instance AWS EC2 menggunakan Nginx sebagai web server, sehingga antarmuka aplikasi dapat diakses secara publik melalui jaringan internet.
- Deployment backend berhasil dilakukan pada AWS ECS dengan launch type Fargate, yang memungkinkan container berjalan secara serverless dan terkelola tanpa perlu mengelola infrastruktur server secara langsung.
- Seluruh komponen sistem telah terintegrasi dengan baik, mulai dari antarmuka pengguna, REST API backend, penyimpanan data SQLite, hingga infrastruktur cloud AWS.

Implementasi ini membuktikan bahwa konsep komputasi awan (cloud computing) dapat diterapkan secara nyata dalam pengembangan aplikasi web modern. Penggunaan teknologi container (Docker) bersama layanan cloud AWS memberikan keuntungan berupa portabilitas, skalabilitas, isolasi lingkungan, dan kemudahan dalam proses deployment serta pemeliharaan aplikasi.

BAB X

TUTORIAL PEMBUATAN APLIKASI

Bab ini mendokumentasikan langkah-langkah teknis pembuatan aplikasi VitalTrack BMI secara lengkap, mulai dari persiapan lingkungan pengembangan hingga proses deployment ke infrastruktur cloud AWS. Tutorial ini dapat digunakan sebagai panduan untuk mereplikasi atau mengembangkan aplikasi serupa.

10.1 Persiapan Lingkungan Pengembangan

Sebelum memulai proses pengembangan, pastikan seluruh perangkat lunak berikut telah terinstal pada komputer pengembang:

Perangkat Lunak	Versi	Fungsi
Node.js	v18.x atau lebih baru	Runtime JavaScript untuk menjalankan backend
npm	v8.x atau lebih baru	Package manager untuk menginstal dependency
Visual Studio Code	Versi terbaru	Code editor dengan dukungan JavaScript
Docker Desktop	v24.x	Platform containerization (opsional di Windows)
Git	v2.x	Version control untuk mengelola kode sumber
Akun AWS	-	Akses ke layanan EC2 dan ECS
Akun Docker Hub	-	Registry untuk menyimpan Docker image

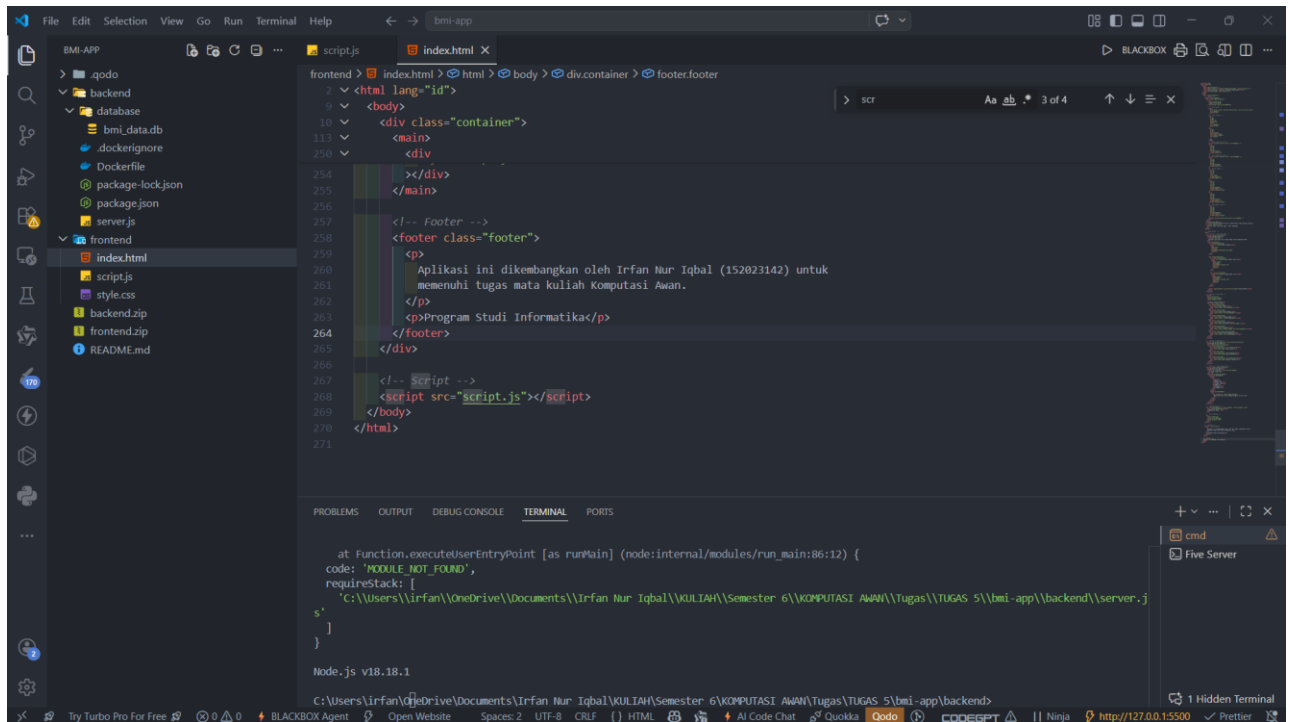
10.2 Pembuatan Struktur Project

Struktur folder project dirancang agar memisahkan komponen frontend dan backend secara jelas. Buat struktur direktori berikut:

bmi-app/

```
|— backend/
| |— server.js      # File utama backend
| |— package.json   # Konfigurasi Node.js
| |— package-lock.json # Lock file dependency
| |— Dockerfile     # Konfigurasi Docker
| |— .dockerignore  # File yang dikecualikan dari Docker
| |— database/      # Direktori penyimpanan SQLite
```

- └─ frontend/
 - ├─ index.html # Halaman utama
 - ├─ style.css # Styling antarmuka
 - └─ script.js # Logika JavaScript frontend



10.3 Pembuatan Backend Node.js

Langkah 1 — Inisialisasi project Node.js. Buka terminal, masuk ke direktori backend, dan jalankan perintah berikut:

```
cd bmi-app/backend
npm init -y
```

Langkah 2 — Instal dependency yang diperlukan:

```
npm install express cors sqlite3
```

Langkah 3 — Buat file server.js dengan endpoint API berikut:

- GET /health: endpoint health check yang mengembalikan status server.
- POST /data: menerima data nama, berat, dan tinggi; menghitung BMI; menyimpan ke database; mengembalikan hasil analisis.
- GET /data: mengambil seluruh riwayat data beserta statistik (total data, rata-rata BMI, jumlah normal).
- DELETE /data: menghapus seluruh data dari database.

Langkah 4 — Verifikasi backend berjalan dengan perintah:

```
npm start
```

Server akan aktif di `http://localhost:3000` dan menampilkan pesan konfirmasi di terminal.

10.4 Pembuatan Database SQLite

Database SQLite dibuat secara otomatis oleh backend saat pertama kali server dijalankan. Proses pembuatan tabel dilakukan melalui kode Node.js menggunakan query SQL berikut yang dieksekusi di dalam callback koneksi database:

```
db.run(`
  CREATE TABLE IF NOT EXISTS bmi_records (
    id          INTEGER PRIMARY KEY AUTOINCREMENT,
    name        TEXT      NOT NULL,
    weight      REAL      NOT NULL,
    height      REAL      NOT NULL,
    bmi         REAL      NOT NULL,
    category    TEXT      NOT NULL,
    created_at  TEXT      NOT NULL
  ) `
);
```

File database `bmi_data.db` akan dibuat secara otomatis di dalam direktori `backend/database/` pada saat server pertama kali dijalankan.

10.5 Pembuatan Frontend

Langkah 1 — Buat file `index.html` yang memuat komponen antarmuka berikut:

- Header dengan nama aplikasi dan identitas pembuat.
- Form input dengan kolom nama, berat badan (kg), dan tinggi badan (cm).
- Seksi tampilan hasil analisis BMI (tersembunyi saat pertama kali dibuka).
- Kartu statistik ringkasan (total data, rata-rata BMI, jumlah Normal).
- Tabel riwayat data dengan kolom No, Nama, Berat, Tinggi, BMI, Kategori, dan Tanggal.

Langkah 2 — Buat file `style.css` dengan desain modern yang mencakup gradient background, card layout, dan badge kategori berwarna.

Langkah 3 — Buat file `script.js` dengan fungsi-fungsi utama berikut:

- `checkHealth()`: memeriksa ketersediaan backend saat halaman dimuat.
- `submitBMI()`: mengirim data form ke backend melalui POST `/data`.
- `loadAllData()`: mengambil riwayat data dari GET `/data` dan memperbarui tampilan.
- `displayResult()`: menampilkan hasil analisis BMI ke antarmuka.
- `updateTable()`: memperbarui tabel riwayat dengan data terbaru.
- `updateStatistik()`: memperbarui kartu statistik.

Langkah 4 — Pastikan variabel `API_BASE_URL` ditetapkan di bagian atas `script.js`:


```
const API_BASE_URL = "http://localhost:3000"; // Ganti saat deploy
```

Langkah 5 — Uji aplikasi secara lokal dengan membuka index.html di browser (gunakan Live Server VS Code untuk menghindari CORS issue) dan pastikan seluruh fitur berfungsi.

10.6 Pembuatan Docker Container

Langkah 1 — Buat file Dockerfile di dalam direktori backend/:

```
FROM node:18-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install --production
COPY . .
RUN mkdir -p database
EXPOSE 3000
ENV NODE_ENV=production
HEALTHCHECK --interval=30s --timeout=3s --start-period=5s --retries=3 \
  CMD node -e "require('http').get('http://localhost:3000/health',
    (r) => { process.exit(r.statusCode === 200 ? 0 : 1); }).on('error',
    () => process.exit(1))"
CMD ["node", "server.js"]
```

Langkah 2 — Buat file .dockerignore untuk mengecualikan file yang tidak perlu dari image:

```
node_modules
npm-debug.log
.git
.env
.vscode
```

Langkah 3 — Upload project ke GitHub (tanpa folder node_modules) dan clone ke server EC2:

```
git clone https://github.com/USERNAME/bmi-app.git
cd bmi-app/backend
```

Langkah 4 — Build Docker image di server EC2:

```
sudo docker build -t bmi-backend .
```

Langkah 5 — Jalankan container untuk pengujian:

```
sudo docker run -d -p 3000:3000 --name bmi-api bmi-backend
```

Langkah 6 — Verifikasi container berjalan dengan status healthy:

```
sudo docker ps
```

10.7 Push Image ke Docker Hub

Setelah image berhasil dibuat dan diverifikasi, langkah selanjutnya adalah mempublikasikannya ke Docker Hub:

Langkah 1 — Login ke Docker Hub dari terminal EC2:

```
sudo docker login -u irfannniqballl DOCKER_HUB
# Masukkan Access Token (bukan password) saat diminta
```

Langkah 2 — Berikan tag pada image sesuai dengan username dan nama repository:

```
sudo docker tag bmi-backend irfannniqballl/bmi-backend:v1
```

Langkah 3 — Push image ke Docker Hub:

```
sudo docker push irfannniqballl/bmi-backend:v1
```

Setelah proses push selesai, image akan tersedia di halaman Docker Hub dan dapat digunakan oleh layanan AWS ECS.

10.8 Deployment Frontend ke EC2

Langkah 1 — Buat instance EC2 baru di AWS Console dengan spesifikasi:

- AMI: Ubuntu Server 22.04 LTS
- Instance type: t2.micro
- Security Group: buka port 22 (SSH), 80 (HTTP), dan 3000 (Custom TCP)

Langkah 2 — Hubungkan ke instance menggunakan EC2 Instance Connect di browser AWS Console.

Langkah 3 — Instal dan aktifkan Docker di EC2:

```
sudo apt update
sudo apt install docker.io -y
sudo systemctl start docker
sudo systemctl enable docker
```

Langkah 4 — Instal Nginx untuk melayani file frontend:

```
sudo apt install nginx -y
sudo systemctl start nginx
sudo systemctl enable nginx
```

Langkah 5 — Clone project dan salin file frontend ke direktori Nginx:

```
git clone https://github.com/USERNAME/bmi-app.git
cd bmi-app/frontend
# Edit API_BASE_URL di script.js sesuai IP EC2
sudo cp index.html style.css script.js /var/www/html/
```

Langkah 6 — Restart Nginx dan akses aplikasi melalui IP publik EC2:

```
sudo systemctl restart nginx  
# Buka browser: http://IP_PUBLIK_EC2
```

10.9 Deployment Backend ke ECS

Langkah 1 — Buka AWS Console, navigasi ke layanan Elastic Container Service (ECS).

Langkah 2 — Buat Cluster baru:

- Klik "Create Cluster" → Pilih "Networking only (Fargate)"
- Beri nama: bmi-cluster → Klik Create

Langkah 3 — Buat Task Definition:

- Navigasi ke Task Definitions → Create new Task Definition
- Launch type: Fargate | OS: Linux/X86_64
- CPU: 0.25 vCPU | Memory: 0.5 GB
- Klik "Add Container" dan isi: nama container: bmi-backend
- Image URI: NAMA_AKUN/bmi-backend:v1
- Port mappings: 3000 TCP
- Klik Create

Langkah 4 — Jalankan Task:

- Masuk ke Cluster bmi-cluster → Tab Tugas → Klik "Jalankan Tugas"
- Launch type: Fargate | Task Definition: bmi-backend
- Networking: Pilih subnet yang tersedia
- Security Group: Buat baru dengan aturan Custom TCP Port 3000 Source 0.0.0.0/0
- IP Publik: Aktifkan (Enable)
- Klik "Buat/Run" dan tunggu status berubah menjadi RUNNING

Langkah 5 — Verifikasi dengan mengakses endpoint health check menggunakan Public IP yang diperoleh dari detail task ECS:

```
| http://PUBLIC_IP_ECS:3000/health
```

10.10 Verifikasi dan Pengujian Akhir

Setelah seluruh komponen berhasil dideploy, lakukan pengujian akhir untuk memastikan integrasi sistem berjalan dengan baik:

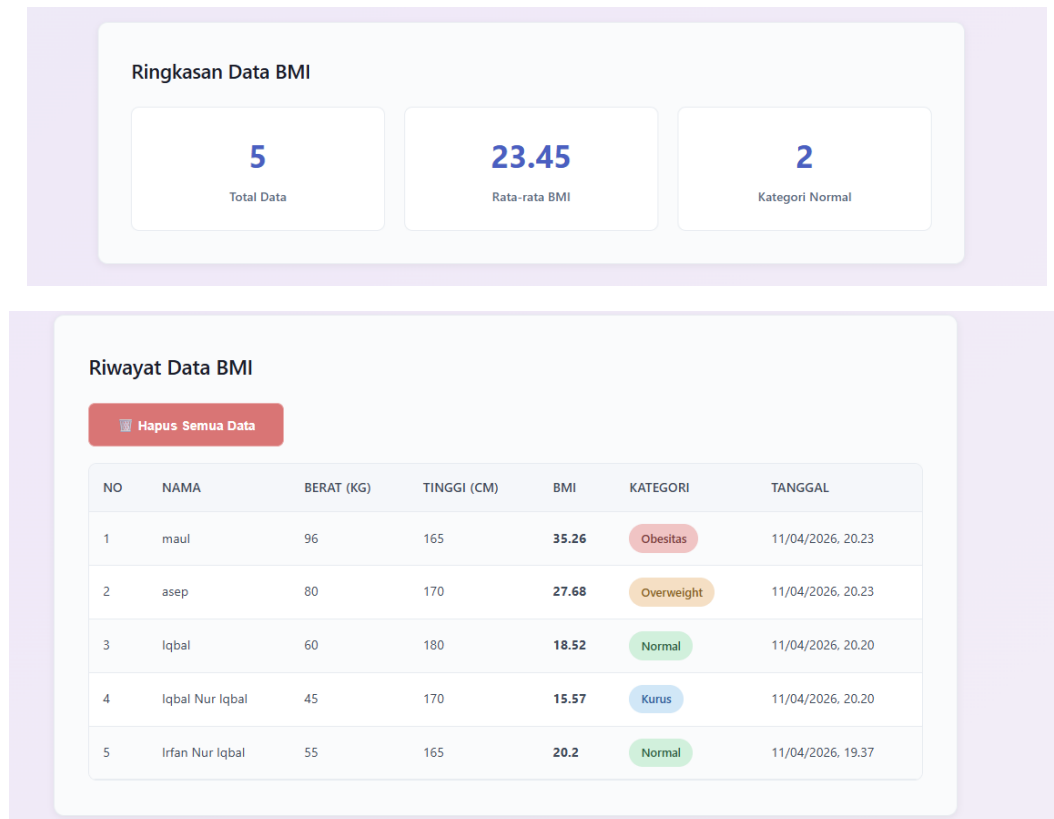
1. Buka frontend melalui IP publik EC2 di browser.
2. Isi form dengan data nama, berat badan, dan tinggi badan.
3. Klik tombol submit dan verifikasi hasil BMI serta kategori muncul.
4. Periksa tabel riwayat data untuk memastikan data tersimpan.
5. Periksa statistik ringkasan untuk memastikan perhitungan akurat.
6. Akses endpoint /health untuk mengkonfirmasi backend ECS aktif.

The screenshot shows a web browser window with the URL 13.214.182.198. The page title is "VitalTrack BMI" and the subtitle is "Sistem Analisis Indeks Massa Tubuh Berbasis Web". Below the title, it says "Disusun oleh: Irfan Nur Iqbal | NIM: 152023142". The main content area is a form titled "Hitung BMI Anda" with the instruction "Masukkan data berat dan tinggi badan untuk menghitung BMI". The form has three input fields: "Nama Lengkap" (with placeholder "Masukkan nama Anda"), "Berat Badan (kg)" (with placeholder "Contoh: 70"), and "Tinggi Badan (cm)" (with placeholder "Contoh: 170"). There is a blue button labeled "Hitung BMI" at the bottom of the form.

The screenshot shows the "Hasil Analisis BMI" section of the web application. It displays the results of the BMI calculation for a user named "Iqbal". The results are shown in a table-like format with the following data:

Nama	Iqbal
Berat Badan	60 kg
Tinggi Badan	180 cm
Nilai BMI	18.52
Kategori	Normal

Below the table, there is a section titled "Pesan Kesehatan" with the message "Indeks massa tubuh berada pada rentang ideal."



Gambar 10.2 Hasil Pengujian Akhir — Aplikasi Berjalan Penuh dengan Integrasi Cloud

Apabila seluruh langkah di atas berhasil dilaksanakan, maka aplikasi VitalTrack BMI telah berhasil diimplementasikan secara lengkap sebagai aplikasi cloud-native dengan containerization Docker dan deployment AWS EC2 + ECS.